МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий

Кафедра квантовых информационных технологий,
практической и прикладной информатики

Отчет по практическим работам №5-8
«Бронирование бильярда»

по дисциплине «Микросервисная архитектура»

Выполнили:
Студенты группы ИКБО-63-23 Черновасиленко Д. С.
Исаченко А. Д.
Волентир А. Ф.
Нартаджиев А. Р.

Проверил:
Старший преподаватель Вохрин О. Л.

Москва, 2026 г.

Состав работы и распределение участников

В работе реализована система бронирования столов бильярдного клуба, спроектированная в практических работах 1-4. Разработка включает обмен сообщениями через RabbitMQ, проектирование HTTP API, создание восьми микросервисов и их тестирование.

Система позволяет выбрать стол и время, получить стоимость, оплатить и отменить бронь, учесть фактическое посещение и подготовить уведомления. Каждый участник отвечает за два сервиса и согласует их контракты с остальными.

Участник	Сервисы	Зона ответственности
Черновасиленко Д. С.	S1 Пользователь S8 Уведомления Python	Профиль клиента, контакты, сообщения и напоминания.
Исаченко А. Д.	S2 Каталог столов S5 Тарифы Python	Описание столов, виды игры, ставки и расчёт стоимости.
Волентир А. Ф.	S3 Расписание S7 Игровые сеансы Node.js	Занятость столов, временный резерв и фактическое посещение.
Нартаджиев А. Р.	S4 Бронирование S6 Платежи Go	Жизненный цикл заявки, оплата и возврат.

Пары сервисов сохранены из предыдущего отчёта; участники сопоставлены порядку фамилий на его титульном листе. Для индивидуального задания 5 используются варианты 1, 3, 2 и 4 соответственно.

Исходный код и электронные материалы: https://github.com/weirdflexer/microservices.

СОДЕРЖАНИЕ

Практическая работа 5 Брокер сообщений.....3

Практическая работа 6 Проектирование REST API.....9

Практическая работа 7 Создание микросервисов.....19

Практическая работа 8 Тестирование микросервисов.....25

Вывод.....29

Использованные материалы.....30

Практическая работа 5 Брокер сообщений

Цель работы

Закрепить навыки асинхронного межсервисного взаимодействия с помощью RabbitMQ и реализовать индивидуальные варианты producer и consumer.

Индивидуальные варианты

Номер варианта определяется остатком от деления последней цифры студенческого билета на 4. Нулевой остаток соответствует варианту 4. У Анвара и Андрея получился одинаковый вариант; по согласованию Андрей выбрал свободный вариант 3.

Таблица 1 - Параметры индивидуальных вариантов

Участник	Цифра / вариант	Очередь	Exchange	Задержка в секундах
Черновасиленко Д. С.	1 / 1	Auto-delete	Topic	Число точек в теле сообщения
Исаченко А. Д.	8 / 3	Auto-delete	Fanout	Случайное целое от 1 до 5
Волентир А. Ф.	2 / 2	Durable	Direct	Длина текста сообщения
Нартаджиев А. Р.	8 / 4	Durable	Direct	Число, переданное в теле

Правила выполнения

Имя очереди имеет вид <группа>_<фамилия>. Для вариантов 2 и 4 сохраняются очередь, exchange и сообщения; для вариантов 1 и 3 очередь автоматически удаляется после отключения последнего consumer. Все профили хранятся в practice5/profiles.json.

Длина считается в Unicode-символах после декодирования UTF-8. Например, «Стол» даёт 4 секунды, «Бронь...» - 3 секунды для варианта 1. Вариант 4 принимает конечное неотрицательное число, включая дробное; неверные значения отклоняются.

Для временных неэксклюзивных очередей в RabbitMQ 4.3+ включена настройка совместимости deprecated_features.permit.transient_nonexcl_queues. Очереди бизнес-сервисов являются durable и от неё не зависят.

Отправка и приём сообщений

Обе программы объявляют одинаковую очередь и привязку. Producer публикует сообщение с mandatory и publisher confirms. Consumer получает одно сообщение за раз, имитирует нагрузку и подтверждает обработку после задержки.

Настройка отправителя

```
channel.confirm_delivery()
channel.basic_publish(
    exchange=exchange,
    routing_key=routing_key,
    body=args.message.encode("utf-8"),
    properties=pika.BasicProperties(
        delivery_mode=2 if profile.durable else 1,
        message_id=str(uuid4()),
    ),
    mandatory=True,
)
```

Обработчик получателя

```
channel.basic_qos(prefetch_count=1)
# Фрагмент обработки корректного сообщения
message = body.decode("utf-8")
delay = delay_seconds(profile.variant, message)
connection.sleep(delay)
channel.basic_ack(delivery_tag=method.delivery_tag)
```

BlockingConnection.sleep обслуживает heartbeat во время ожидания. Неверное число или кодировка приводят к nack(requeue=False). Подтверждение издателю означает приём брокером и не заменяет ACK от consumer.

Пример запуска

```
python -m practice5.consumer --student danek
python -m practice5.producer --student danek 'Бронь...'
# Встроенный exchange вместо варианта:
python -m practice5.consumer --student sasha --mode simple
```

В режиме simple используется exchange с пустым именем и routing key, равный имени очереди. В режиме variant используются topic/direct/fanout. Два consumer одной очереди конкурируют; fanout копирует сообщение в разные привязанные очереди.

Топология RabbitMQ



Рисунок 1 - Маршрутизация событий бронирования

Таблица 2 - Индивидуальные очереди

Профиль	Очередь	Binding
danek	ikbo-63-23_chernovasilenko	billiards.*
andrey	ikbo-63-23_isachenko	Пустой; fanout игнорирует ключ
sasha	ikbo-63-23_volentir	billiards.demo
anvar	ikbo-63-23_nartadzhiev	billiards.demo

Имя индивидуального exchange образуется добавлением practice5. к имени очереди. Для topic и direct публикация использует ключ billiards.demo; для fanout ключ пустой.

Бизнес-система использует отдельный durable topic exchange billiards.events и очереди billiards.bookings, billiards.payments, billiards.schedule, billiards.sessions и billiards.notifications. Отдельная очередь каждого потребителя позволяет независимо обрабатывать копии одного события.

Очередь billiards.audit подписана на # и ограничена 10 000 сообщениями. У каждого потребителя есть очередь ошибок billiards.<service>.dead, связанная с direct exchange billiards.dead. S1, S2 и S5 получают бизнес-запросы синхронно.

Формат доменного сообщения

Все сообщения передаются как JSON UTF-8. Общая оболочка позволяет определить тип события, связать его с процессом бронирования и безопасно распознать повторную доставку.

Таблица 3 - Поля оболочки события

Поле оболочки	Тип	Назначение
`id`	UUID-строка	Идентификатор события; не меняется при повторной доставке
`type`	строка	Тип события, совпадает с routing key
`occurred_at`	ISO 8601, с часовым поясом	Время факта, в UTC
`producer`	строка	Имя публикующего сервиса
`correlation_id`	UUID-строка или null	Для процесса бронирования равен `booking_id`
`data`	объект	Тело по каталогу ниже

Пример события подтверждения

```
{
  "id": "a518fe1b-6aeb-4b71-90ad-203b107ca46c",
  "type": "booking.confirmed",
  "occurred_at": "2026-11-06T18:00:00+00:00",
  "producer": "bookings",
  "correlation_id": "4b849709-dace-4d04-8175-ef3fe66170a9",
  "data": {
    "booking_id": "4b849709-dace-4d04-8175-ef3fe66170a9",
    "user_id": "7e13d2ae-5d40-4a6b-bcb8-22a4c1cb4cdd",
    "table_id": "f28c26c5-43cf-432a-ad3e-8300f34f7d95",
    "start_at": "2026-11-07T14:00:00+00:00",
    "end_at": "2026-11-07T15:00:00+00:00",
    "amount": 100000,
    "currency": "RUB"
  }
}
```

Все поля *_id содержат UUID-строки, а не копии чужих сущностей. Денежные суммы передаются целым числом копеек. Время содержит часовой пояс. payment_url указывает относительный путь учебного callback в S6; reason поясняет причину возврата.

Каталог событий оформления и подтверждения

Ключ маршрутизации совпадает с type сообщения. В таблице приведены издатели, подписчики, обязательные поля data и выполняемые действия.

Таблица 4 - События оформления оплаты и отмены

Routing key	Издатель → подписчики	Поля data	Выполняемое действие
booking.payment_requested	S4 bookings → S6 payments	booking_id, user_id, table_id, start_at, end_at, amount, currency, expires_at	Создать один платёж на бронь и получить данные оплаты
payment.created	S6 payments → S4 bookings	booking_id, payment_id, payment_url	Сохранить идентификатор и ссылку оплаты
payment.succeeded	S6 payments → S4 bookings	booking_id, user_id, payment_id, amount, currency	Проверить сумму, синхронно закрепить действующее удержание в S3, затем подтвердить бронь; для поздней оплаты инициировать возврат
payment.failed	S6 payments → S4 bookings	booking_id, user_id, payment_id, amount, currency	Учесть неуспешную попытку оплаты; бронь не считать оплаченной
booking.confirmed	S4 bookings → S7 sessions, S8 notifications	booking_id, user_id, table_id, start_at, end_at, amount, currency	S7: создать ожидаемое посещение; S8: подготовить подтверждение и напоминание
booking.cancelled	S4 bookings → S3 schedule, S7 sessions, S8 notifications	booking_id, user_id, table_id, start_at, end_at, amount, currency, hold_id	S3: освободить интервал; S7: отменить ожидаемое посещение; S8: снять неотправленное напоминание
booking.expired	S4 bookings → S3 schedule, S7 sessions, S8 notifications	booking_id, user_id, table_id, start_at, end_at, amount, currency, hold_id	S3: освободить оставшийся интервал; S7: отменить ожидаемое посещение, если оно существовало; S8: снять напоминание

Связи отмены и истечения с S8 добавлены для снятия неотправленных напоминаний. Событие booking.expired также поступает в S3 для освобождения интервала, если процесс закрыт самим S4.

Каталог событий возврата и посещения

Таблица 5 - Возврат, истечение удержания и завершение посещения

Routing key	Издатель → подписчики	Поля data	Выполняемое действие
booking.refund_requested	S4 bookings → S6 payments	booking_id, user_id, payment_id, amount, currency, reason	Запросить возврат, не превышающий полученную сумму; повтор не создаёт второй возврат
payment.refunded	S6 payments → S4 bookings, S8 notifications	booking_id, user_id, payment_id, amount, currency	S4: отметить завершённый возврат; S8: подготовить сообщение клиенту
payment.refund_failed	S6 payments → S4 bookings	booking_id, user_id, payment_id, amount, currency	Зафиксировать незавершённое обязательство; не объявлять деньги возвращёнными
schedule.hold_expired	S3 schedule → S4 bookings	booking_id, hold_id	Закрыть ещё неоплаченную бронь; устаревшее сообщение не меняет подтверждённую бронь
session.completed	S7 sessions → S4 bookings	booking_id, session_id	Завершить подтверждённую бронь после окончания игры
session.no_show	S7 sessions → S4 bookings	booking_id, session_id	Закрыть бронь по неявке, не выполнять автоматический возврат

Подтверждения и повторная доставка

Изменение состояния и запись outbox сохраняются одной локальной транзакцией. После publisher confirm событие удаляется из outbox. Если процесс остановился между публикацией и удалением, возможна повторная доставка с тем же id.

Consumer сохраняет id в inbox вместе с бизнес-изменением и только затем отправляет ACK. Дубликат подтверждается без повторного действия. При временной ошибке допускаются три попытки; некорректное сообщение или окончательный отказ попадает в DLQ. Публикация повтора или DLQ подтверждается до ACK исходной доставки.

В индивидуальном задании выполнены восемь проверок: четыре профиля в режимах simple и variant. Во всех случаях подтверждены отправка, задержка и ACK. Полный журнал хранится в reports/rabbitmq.txt.

Практическая работа 6 Проектирование REST API

Цель работы

Спроектировать HTTP API всех синхронных команд восьми микросервисов и определить назначение параметров, структуру запросов, ответов и ошибок.

Общие соглашения

Пути начинаются с /api. Тела запросов и ответов имеют формат JSON UTF-8. Идентификаторы передаются UUID-строками. Время - ISO 8601 с часовым поясом; хранение и ответы нормализованы к UTC. Суммы amount и hourly_rate задаются целыми копейками, валюта RUB.

Интервал [start_at, end_at) полуоткрытый: соседние интервалы не пересекаются. Начало кратно 30 минутам; длительность от 1 до 4 часов с шагом 30 минут. При создании брони или предложения начало должно быть в будущем.

Списки возвращаются JSON-массивом. offset - число пропускаемых элементов, по умолчанию 0; limit - размер страницы от 1 до 100. Значение limit по умолчанию 50 у S1/S2/S5, 100 у S4/S6/S7/S8. Фильтры применяются до пагинации.

Таблица 6 - Коды HTTP-ответов

Код	Назначение
200 / 201	Операция выполнена / ресурс создан
401	Неверный токен учебного платёжного callback
404	Ресурс не найден
409	Конфликт состояния, занятость, дубликат или истёкший срок
422	Неверная структура, тип поля, UUID или время
503	Зависимый сервис недоступен или вернул серверную ошибку

Доменная ошибка имеет вид {"detail":"Интервал уже занят"}. Ошибки структуры возвращают detail; форма описания зависит от HTTP-адаптера языка. Обычные бизнес-эндпоинты учебного стенда работают без авторизации. Callback S6 требует X-Provider-Token; локальное значение demo-provider-token.

Каждый сервис публикует /docs и /openapi.json. Технический /health показывает состояние процесса и broker_connected. Эти маршруты не учитываются в требовании 2-5 бизнес-эндпоинтов.

S1 Пользователь

Базовый адрес: http://localhost:8001

Метод и путь	Назначение	Запрос	Ответ
`POST /api/users`	Создать клиента	`name`, `contact`	201, `User`
`GET /api/users`	Получить клиентов	query `offset`, `limit`	200, `User[]`
`GET /api/users/{user_id}`	Получить профиль и контакт	UUID пользователя в пути	200, `User`; 404
`PATCH /api/users/{user_id}`	Изменить профиль/контакт	хотя бы одно из `name`, `contact`	200, `User`; 404/409/422

Поле `User`	Тип и смысл
`id`	UUID клиента, возвращается сервером
`name`	Имя, 1–100 символов, пробелы по краям удаляются
`contact`	Email или телефон, 3–254 символа; уникален без учёта регистра
`created_at`	Время регистрации

При создании обязательны `name`, `contact`. В PATCH не переданное поле сохраняется, явный `null` и пустой объект запрещены. Дубликат контакта — 409. Проверка формата телефона/email ограничена длиной; нормализация телефонов к международному виду не реализована.

```
{"name":"Иван","contact":"ivan@example.test"}
```

Ответ содержит эти же поля и созданные `id`, `created_at`.

S2 Каталог столов

Базовый адрес: http://localhost:8002

Метод и путь	Назначение	Запрос	Ответ
`POST /api/tables`	Добавить стол	`number`, `game_type`, необязательные `description`, `published`	201, `Table`; 409 при повторе номера
`GET /api/tables`	Подобрать стол	query `offset`, `limit`, `published?`, `game_type?`	200, `Table[]`
`GET /api/tables/{table_id}`	Получить сведения	UUID стола	200, `Table`; 404
`PATCH /api/tables/{table_id}`	Изменить стол	любое непустое подмножество изменяемых полей	200, `Table`; 404/409/422

Поле `Table`	Тип и смысл
`id`	UUID стола
`number`	Положительный уникальный номер в клубе
`game_type`	`pool`, `russian` или `snooker`
`description`	Текст до 1000 символов, по умолчанию пустая строка
`published`	Boolean, по умолчанию false; true разрешает бронирование

Публикация — `PATCH` с `{"published":true}`. Занятость в карточке не хранится. PATCH не принимает `null` и пустой объект. Фильтр `published=true` вернёт только опубликованные столы; значение `game_type` имеет те же варианты, что и поле объекта.

S3 Расписание

Базовый адрес: http://localhost:8003

Метод и путь	Назначение	Запрос	Ответ
`PUT /api/tables/{table_id}/schedule`	Задать рабочее окно	`opens_at`, `closes_at`	200, `Schedule`; проверка стола через S2
`GET /api/tables/{table_id}/availability`	Проверить свободный интервал	обязательные query `start_at`, `end_at`	200, `Availability`; 404 без графика
`POST /api/holds`	Удержать интервал	`booking_id`, `table_id`, `start_at`, `end_at`	201, `Hold`; 409 при занятости
`POST /api/holds/{hold_id}:confirm`	Закрепить удержание	`booking_id`	200, `Hold`; 409 если срок истёк/чужая бронь
`DELETE /api/holds/{hold_id}`	Освободить интервал	обязательный query `booking_id`, тела нет	200, `Hold`; 404/409

`Schedule`: `table_id`, `opens_at`, `closes_at`. Рабочее окно задаётся абсолютными моментами времени; повторяющийся недельный календарь не моделируется. Закрытие должно быть позже открытия, новый график не может вытеснить действующие удержания/брони.

`Hold`: `id` — UUID удержания; `booking_id` — его владелец; `table_id` — стол; `start_at`, `end_at` — игровой интервал; `expires_at` — срок удержания; `status` — `held`, `confirmed`, `released` или `expired`. Срок равен минимуму из текущего времени +10 минут и начала игры. Повтор запроса удержания с тем же `booking_id` и тем же активным интервалом возвращает существующее удержание. Другие данные для этой же брони дают 409. Повторное закрепление уже закреплённого интервала и повторное освобождение безопасны.

`Availability`: `table_id`, `start_at`, `end_at`, `available` (boolean) и `busy_intervals` (`Hold[]` с пересечениями). `available=false` также получается для прошедшего времени или интервала вне графика. Чтение доступности ничего не резервирует; победитель при конкуренции определяется транзакцией `POST /api/holds`.

S4 Бронирование

Базовый адрес: http://localhost:8004

Метод и путь	Назначение	Запрос	Ответ
`POST /api/bookings`	Создать бронь с ожиданием оплаты	`user_id`, `table_id`, `quote_id`, `start_at`, `end_at`	201, `Booking`; проверки S1/S5/S3
`GET /api/bookings`	Найти брони	query `user_id?`, `status?`, `offset`, `limit`	200, `Booking[]`
`GET /api/bookings/{booking_id}`	Получить состояние и оплату	UUID брони	200, `Booking`; 404
`POST /api/bookings/{booking_id}:cancel`	Отменить до начала	UUID брони, тела нет	200, `Booking`; 404/409

Поле `Booking`	Тип и смысл
`id`	UUID брони
`user_id`, `table_id`, `quote_id`, `hold_id`	Ссылки на клиента, стол, предложение, удержание
`start_at`, `end_at`	Зафиксированный игровой интервал
`amount`, `currency`	Зафиксированная стоимость и `RUB`
`expires_at`	Срок оформления, полученный от S3
`status`	`waiting_payment`, `confirmed`, `cancelled`, `expired`, `completed`, `no_show`; `rejected` зарезервирован для отклонённого состояния
`payment_id`, `payment_url`	До события `payment.created` равны null; затем реквизиты оплаты
`paid`	Получено ли событие успешной оплаты
`refund_status`	null, `pending`, `succeeded` или `failed`
`payment_attempts`	Список учтённых исходов неуспешной оплаты
`created_at`	Время создания
`cancelled_at`	Время отмены, до отмены null

201 означает создание заявки и ожидание оплаты. Клиент повторяет GET, чтобы получить созданный асинхронно платёж и итог подтверждения. S4 не принимает от клиента сумму: она получается из проверенного предложения S5. После успешной оплаты S4 закрепляет удержание в S3 и лишь затем меняет статус на `confirmed`.

Отмена повторяемая: уже отменённая бронь возвращается без второго события. Отмена допускается до начала, при отмене оплаченной брони за два часа или более до начала запускается возврат. Смена статуса оплаты, закрытие по сроку и завершение брони не имеют публичных ручных эндпоинтов: ими управляют события. Поздняя оплата ранее неоплаченной закрытой брони вызывает полный возврат.

S5 Тарифы и ценовые предложения

Базовый адрес: http://localhost:8005

Метод и путь	Назначение	Запрос	Ответ
`POST /api/tariffs`	Опубликовать новую версию	`game_type`, `hourly_rate`	201, `Tariff`
`GET /api/tariffs`	Получить версии	query `game_type?`, `offset`, `limit`	200, `Tariff[]`
`POST /api/quotes`	Рассчитать стоимость	`table_id`, `start_at`, `end_at`	201, `Quote`; проверка S2
`POST /api/quotes/{quote_id}:validate`	Проверить применимость	те же `table_id`, `start_at`, `end_at`	200, сохранённый `Quote`; 404/409

`Tariff`: `id` — UUID версии; `game_type` — вид игры; `hourly_rate` — положительная цена часа в копейках; `version` — последовательный номер версии для этого вида игры; `currency` — `RUB`; `published_at` — время публикации. Изменение опубликованной версии не предусмотрено: следующий POST создаёт новую, старые предложения сохраняют свою цену.

`Quote`: `id` — UUID предложения; `table_id`, `start_at`, `end_at` — применимость; `amount`, `currency` — итоговая стоимость; `expires_at` — создание +10 минут; `tariff_id`, `tariff_version`, `hourly_rate` — снимок использованного тарифа. Цена считается как ставка × минуты /60 с округлением вверх до целой копейки.

S5 обращается в S2 за видом игры и публикацией стола. Это добавленная к итоговой диаграмме проверка: вид игры не доверяется клиенту и у владельца S2/S5 появляется обязательный межсервисный вызов из HTTP-эндпоинта. Нет тарифа/стол не опубликован — 409. Проверка quote отклоняет другой стол, интервал и истёкший срок с 409; новая публикация тарифа не изменяет ранее созданное действующее предложение.

S6 Платежи

Базовый адрес: http://localhost:8006

Метод и путь	Назначение	Запрос	Ответ
`GET /api/payments`	Найти платежи	query `booking_id?`, `offset`, `limit`	200, `Payment[]`
`GET /api/payments/{payment_id}`	Получить состояние	UUID платежа	200, `Payment`; 404
`POST /api/payments/{payment_id}:result`	Учебный результат оплаты	`status`, `amount`, `currency`; заголовок `X-Provider-Token`	200, `Payment`; 401/404/409/422
`POST /api/payments/{payment_id}:refund-result`	Учебный результат возврата	`status`; тот же заголовок	200, `Payment`; 401/404/409/422

Платёж создаётся событием `booking.payment_requested`, запрос возврата — `booking.refund_requested`. Поэтому HTTP POST для создания платежа и возврата нет. `status` callback принимает `succeeded` или `failed`. В оплате `amount` обязателен, положителен и совпадает со стоимостью платежа; `currency` по умолчанию `RUB`.

`Payment`: `id`, `booking_id`, `user_id`; `amount`, `currency`; `status` = `pending`, `succeeded` или `failed`; `refund_status` = null/`pending`/`succeeded`/`failed`; `refunded_amount` — фактически учтённый возврат в копейках; `created_at`; `payment_url` — относительный путь callback в **S6**, не HTML-страница; `provider` = `demo`; `refund_reason` содержит причину после запроса возврата, до него null.

Повтор успешного callback не создаёт второй переход или событие. Запоздалый отказ после успеха не отменяет оплату. Результат возврата до запроса возврата даёт 409. В учебной модели возвращается полная сумма, частичные возвраты не реализованы.

```
POST http://localhost:8006/api/payments/{payment_id}:result
Content-Type: application/json
X-Provider-Token: demo-provider-token

{"status":"succeeded","amount":100000,"currency":"RUB"}
```

S7 Игровые сеансы

Базовый адрес: http://localhost:8007

Метод и путь	Назначение	Запрос	Ответ
`GET /api/sessions`	Найти посещения	query `booking_id?`, `offset`, `limit`	200, `Session[]`
`GET /api/sessions/{session_id}`	Получить посещение	UUID посещения	200, `Session`; 404
`POST /api/sessions/{session_id}:start`	Начать игру	UUID посещения, тела нет	200, `Session`; проверка брони в S4; 404/409/503
`POST /api/sessions/{session_id}:complete`	Завершить игру	UUID посещения, тела нет	200, `Session`; 404/409

`Session`: `id`, `booking_id`, `user_id`, `table_id`, `start_at`, `end_at`; `status` = `expected`, `active`, `completed`, `cancelled` или `no_show`; `started_at`, `completed_at` — фактические времена, до соответствующего действия null. Ожидаемое посещение создаётся событием `booking.confirmed`, максимум одно на бронь.

Старт разрешён при `start_at ≤ now < start_at + 15 минут` и актуальной подтверждённой брони с совпадающими клиентом, столом и интервалом. При наступлении 15 минут таймер фиксирует неявку. Повтор старта активного сеанса и завершения уже завершённого безопасен. Завершение неначатого сеанса — 409. В конце интервала таймер завершает активный сеанс.

S8 Уведомления

Базовый адрес: http://localhost:8008

Метод и путь	Назначение	Запрос	Ответ
`POST /api/notifications`	Подготовить сообщение	`user_id`, `message`; необязательные `booking_id`, `kind`, `scheduled_at`	201, `Notification`
`GET /api/notifications`	Найти сообщения	query `user_id?`, `offset`, `limit`	200, `Notification[]`
`GET /api/notifications/{notification_id}`	Получить состояние	UUID уведомления	200, `Notification`; 404
`POST /api/notifications/{notification_id}:send`	Выполнить попытку отправки	UUID уведомления, тела нет	200, `Notification`; 404/409

`Notification`: `id`; `user_id`; `booking_id` (может быть null); `kind` = `custom`, `confirmation`, `reminder` или `refund`, по умолчанию `custom`; `message` — непустой текст 1–2000 символов; `scheduled_at` — время отправки, по умолчанию сейчас; `status` = `prepared`, `sent`, `failed` или `cancelled`; `attempts` — число попыток; `sent_at` — время успеха или null; `last_error` — последняя ошибка или null; `provider` = `demo-log`.

Отправка получает актуальный контакт в S1. Учебный адаптер фиксирует доставку в логе; внешнее сообщение клиенту не отправляется. Будущее `scheduled_at` даёт 409 при ручной попытке раньше срока. Ошибка получения контакта записывается в объект; ответ 200 может содержать `prepared` с отложенной следующей попыткой или `failed`. Следует читать `status` и `last_error`, а не принимать код 200 за факт доставки. Уже отправленное, отменённое или исчерпавшее попытки сообщение повторно не отправляется.

Синхронные связи и пример запроса

Таблица 7 - Синхронные межсервисные вызовы

Сервис	Вызываемый API	Причина
S3	S2 `GET /api/tables/{table_id}`	Проверить существование и публикацию при настройке/удержании
S4	S1 `GET /api/users/{user_id}`	Проверить клиента перед оформлением
S4	S5 `POST /api/quotes/{quote_id}:validate`	Зафиксировать действующую цену выбранного интервала
S4	S3 `POST /api/holds`	Атомарно занять время
S4	S3 `POST /api/holds/{hold_id}:confirm`	После оплаты закрепить удержание
S5	S2 `GET /api/tables/{table_id}`	Получить вид игры для расчёта цены; добавленная связь
S7	S4 `GET /api/bookings/{booking_id}`	Проверить бронь перед стартом
S8	S1 `GET /api/users/{user_id}`	Получить контакт перед доставкой

Создание брони

Клиент сначала получает quote в S5, затем передаёт его идентификатор в S4. Сервер проверяет пользователя, соответствие цены столу и интервалу, после чего запрашивает удержание. Сумма не берётся из тела клиентского запроса.

```
POST http://localhost:8004/api/bookings
Content-Type: application/json

{
  "user_id": "7e13d2ae-5d40-4a6b-bcb8-22a4c1cb4cdd",
  "table_id": "f28c26c5-43cf-432a-ad3e-8300f34f7d95",
  "quote_id": "fd6a8300-a598-4475-8ca8-6cf832d7c501",
  "start_at": "2030-01-15T12:00:00+03:00",
  "end_at": "2030-01-15T14:00:00+03:00"
}
```

Ответ 201 содержит объект Booking со статусом waiting_payment. payment_id и payment_url сначала равны null. Клиент повторяет GET /api/bookings/{booking_id}, чтобы получить реквизиты и затем итог подтверждения. HTTP-ответ о создании брони не означает, что асинхронная оплата уже завершена.

Практическая работа 7 Создание микросервисов

Цель работы

Реализовать выделенные контексты, синхронное и асинхронное взаимодействие, собственное хранение данных каждого сервиса и запуск в отдельных контейнерах.

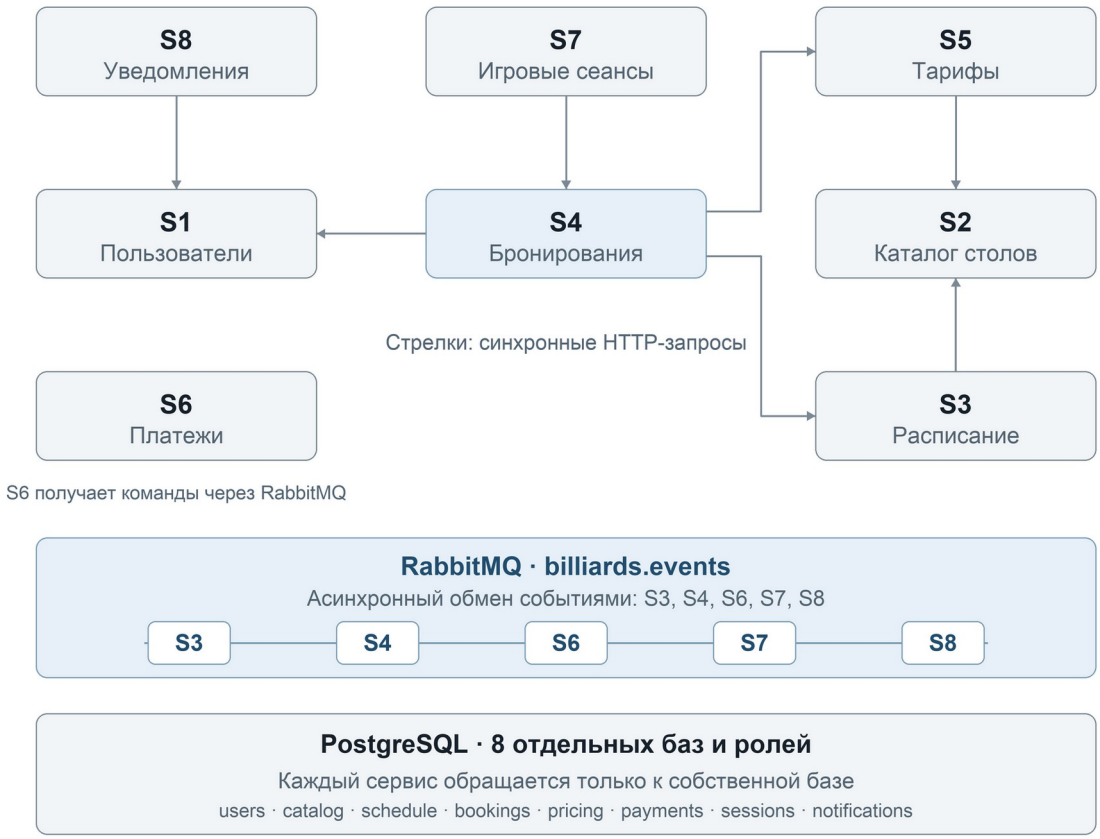


Рисунок 2 - Сервисы и инфраструктура системы

S1/S2/S5/S8 работают на Python: FastAPI, HTTPX, SQLAlchemy/psycopg и Pika. Саша реализует S3/S7 на Node.js: встроенные HTTP/fetch, pg и amqplib. Анвар реализует S4/S6 на Go: net/http, pgx и amqp091-go. Все три языка используют общие HTTP/JSON и AMQP-контракты; внешние провайдеры заменены учебными адаптерами.

Связь S5 → S2 добавлена для проверки публикации стола и получения вида игры при расчёте цены. Таким образом у владельца S2/S5 есть требуемый метод с вызовом другого сервиса. Остальные участники имеют S8 → S1, S3 → S2 / S7 → S4 и S4 → S1/S5/S3.

Организация кода и границы сервисов

Таблица 8 - Структура проекта

Путь	Назначение
`services/<name>/*.py`	S1/S2/S5/S8: api, models, service и main на Python
`services/<name>/*.mjs`	S3/S7: api, service и main на Node.js
`services/<name>/*.go`	S4/S6: HTTP-обработчики и бизнес-логика Go
`services/<name>/cmd/main.go`	Точки запуска Go; сборка бинарного server
`services/<name>/Dockerfile`	Собственный контейнер каждого сервиса
`common/`	Python: Context, SQLStore, HTTP, RabbitMQ и таймеры
`common-node/`	Node.js: репозиторий, валидация, HTTP, AMQP и runtime
`internal/gokit/`	Go: Context, Store, HTTP, AMQP и запуск сервера
`infra/init-databases.sql`	Создание восьми баз и отдельных ролей PostgreSQL
`practice5/`	Четыре индивидуальных варианта RabbitMQ на Python
`tests/unit/`, `tests/component/`	Unit и HTTP-проверки четырёх Python-сервисов
`common-node/tests/`, `services/`	Node.js test-файлы и Go-файлы *_test.go
`tests/integration/`	HTTP-проверки восьми реальных сервисов и перезапуска

Разделение ответственности

HTTP-роутер отвечает за параметры запроса и коды ответа. Сервисный слой содержит правила предметной области. Репозиторий управляет локальной транзакцией. Обработчик RabbitMQ вызывает бизнес-логику при получении события. Контекст передаёт репозиторий, HTTP-адаптер, часы и настройки.

Пакеты common, common-node и internal/gokit содержат технические адаптеры для своих языков. Сервис не читает чужие таблицы и не меняет состояние другого контекста напрямую. Идентификаторы пользователя, стола и брони передаются через опубликованные контракты.

У семи сервисов реализованы четыре бизнес-операции, у расписания пять; всего 33. Точки входа: main.py у Python, main.mjs у Node.js и cmd/main.go у Go. Контейнер запускает соответственно Uvicorn, node или собранный /app/server. Каждый сервис остаётся отдельным процессом.

Хранение данных и локальные транзакции

Таблица 9 - Отдельные базы и роли PostgreSQL

Сервис / база / роль	Основные разделы состояния
`users`	Клиенты, уникальные контакты
`catalog`	Карточки столов, уникальные номера
`schedule`	Рабочие окна, удержания и закреплённые интервалы
`bookings`	Брони, сохранённая цена, статус оплаты/возврата
`pricing`	Неизменяемые версии тарифов, ценовые предложения
`payments`	Платежи, индекс по броням, состояние возвратов
`sessions`	Посещения, индекс по броням, отметки отмены
`notifications`	Сообщения, дедупликация, попытки и отмена

На одном сервере PostgreSQL созданы восемь баз и восемь владельцев. CONNECT для PUBLIC отозван. В каждой базе таблица service_state содержит одну JSON-запись с разделами data, inbox и outbox. Изменение защищено SELECT FOR UPDATE и фиксируется одной транзакцией.

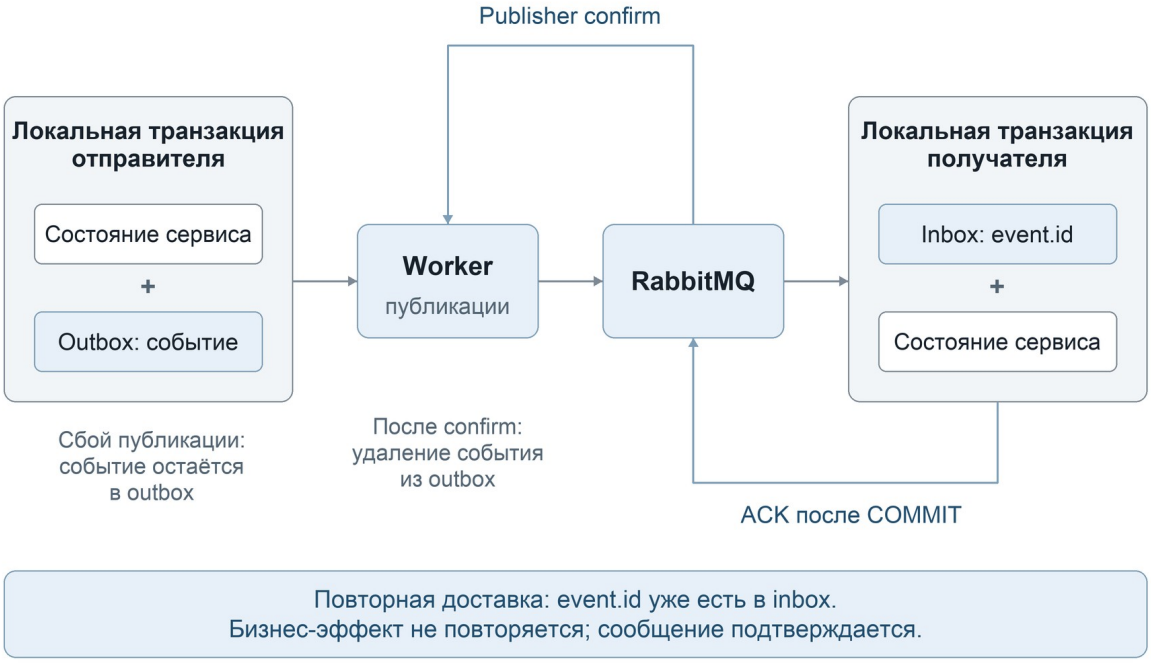


Рисунок 3 - Локальные транзакции и доставка события

Блокировка строки сериализует команды внутри сервиса, в том числе проверку пересечений расписания. Это учебное упрощение: отдельные таблицы агрегатов, ограничения диапазонов и инструмент миграций не реализованы. При росте нагрузки требуется более детальная схема и блокировки отдельных агрегатов.

Жизненный цикл брони и компенсации

Таблица 10 - Основные переходы и бизнес-правила

Событие или команда	Условие	Результат
Создание заявки	Пользователь, quote и интервал корректны	waiting_payment; outbox запрашивает платёж
payment.succeeded	Сумма совпадает, удержание закреплено	confirmed; создаются посещение и уведомление
payment.failed	Бронь ещё не оплачена	Учтена попытка; подтверждения нет
Отмена	До начала игры	cancelled; снятие интервала, посещения и напоминания
Возврат при отмене	Бронь оплачена; до начала не менее 2 часов	Запрос полного возврата; ожидание результата
Поздняя оплата	Ранее неоплаченная бронь уже закрыта	Полный возврат без восстановления брони
schedule.hold_expired	Сообщение относится к текущему удержанию	waiting_payment → expired
session.completed	Бронь подтверждена	completed
session.no_show	Прошло 15 минут; игра не началась	no_show без автоматического возврата

Если ответ S3 о закреплении потерян, S4 повторяет идемпотентный вызов. Временная ошибка вызывает повтор обработки события; окончательный отказ закрепления закрывает бронь и запускает компенсационный возврат. После трёх неудачных попыток событие попадает в DLQ и требует разбора.

Повторный callback уже учтённой оплаты не создаёт второй платёж и не превращает отмену без возврата в основание для позднего возврата. Раннее завершение и неявка не сокращают оплаченный интервал. Таймер удержания не снимает подтверждённую бронь.

Контейнеризация и конфигурация

Compose содержит десять контейнеров: восемь приложений, PostgreSQL и RabbitMQ. Приложения обращаются друг к другу по DNS-именам контейнеров. Порты 8001-8008 и 5672/15672 публикуются только на 127.0.0.1. PostgreSQL не публикуется на хост.

Фрагмент Dockerfile сервиса бронирования на Go

```
FROM golang:1.25-alpine AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY internal/gokit ./internal/gokit
COPY services/bookings ./services/bookings
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" \
    -o /out/server ./services/bookings/cmd
FROM alpine:3.22
RUN apk add --no-cache ca-certificates && adduser -D -u 10001 app
WORKDIR /app
COPY --from=builder /out/server /app/server
USER app
EXPOSE 8000
ENTRYPOINT ["/app/server"]
```

Go запускает бинарный /app/server. Node.js запускает services/<name>/main.mjs; четыре Python-сервиса используют Uvicorn. Зависимости закреплены в go.mod/go.sum, package-lock.json и requirements.txt.

Таблица 11 - Переменные конфигурации

Настройка	Назначение
`DATABASE_URL`	Подключение конкретного сервиса к его БД
`RABBITMQ_URL`	AMQP URL; внутри Compose указывает `rabbitmq:5672`
`PROVIDER_TOKEN`	Токен учебных callback S6
`SERVICE_HOST_TEMPLATE`	Шаблон адресов HTTP, по умолчанию `http://{service}:8000`
`<SERVICE>_URL`	Точное переопределение адреса, например `USERS_URL`
`HTTP_TIMEOUT`	Тайм-аут межсервисных HTTP-запросов, по умолчанию 5 секунд
`TIMER_INTERVAL`	Период фоновой проверки, по умолчанию 1 секунда

Compose использует healthcheck и depends_on для готовности PostgreSQL и RabbitMQ. Данные сохраняются в именованных томах. У каждой базы отдельные учётные данные; значения поставляются для локального учебного стенда.

Запуск системы и границы реализации

Порядок запуска

```
cp .env.example .env
docker compose up -d --build
docker compose ps
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python -m scripts.demo
python -m scripts.check_rabbitmq
```

На macOS проверка выполнена через Podman и podman-compose. В этом случае предварительно запускается Podman machine, а docker compose заменяется на podman-compose. Для нескольких машин можно задать CONTAINER_CONNECTION.

Наблюдение и остановка

Swagger находится на http://localhost:8001/docs ... http://localhost:8008/docs. RabbitMQ Management доступен на http://localhost:15672 с учебными micro / micro. Логи показывают обработку событий; /health возвращает broker_connected. Обычная остановка docker compose down сохраняет данные.

Ограничения

Учебный платёжный провайдер принимает callback и не списывает деньги. Отправка уведомления фиксируется в логе, реальные email/SMS не отправляются. Авторизация пользователей и административные роли не реализованы; токен callback предназначен для демонстрации.

У клиентских создающих запросов нет общего ключа идемпотентности. Если S3 удержал время, а S4 остановился до сохранения заявки, временное удержание освобождается по сроку. Полного журнала шагов саги и автоматической бесконечной сверки зависших процессов нет.

График задан одним абсолютным рабочим окном. История данных и inbox не архивируется. JSON-хранение сериализует команды целого сервиса. Эти ограничения отделены от выполненных требований практики: восемь контейнеров, 2-5 операций, HTTP-вызовы каждого участника и PostgreSQL-хранение.

Практическая работа 8 Тестирование микросервисов

Цель работы

Проверить бизнес-правила и внешние контракты восьми сервисов до развёртывания. Из трёх видов тестирования в методичке выбраны unit и компонентное тестирование.

Изоляция зависимостей

Unit-тесты используют память, подмену HTTP и управляемые часы. Компонентные проверки проходят через реальные HTTP-обработчики: Python TestClient, локальный сервер Node.js с fetch, Go httptest. PostgreSQL и RabbitMQ этим наборам не нужны.

Фрагмент unit-теста конкуренции Node.js

```
const {service, data} = await fixture();
const results = await Promise.allSettled(
  Array.from({length: 32}, () => service.hold({
    ...data, booking_id: randomUUID()
  }))
);
assert.equal(
  results.filter(row => row.status === 'fulfilled').length, 1
);
```

Фрагмент компонентного теста Go

```
f := setup(t)
handler := f.s.Handler(func() bool { return false })
raw, _ := json.Marshal(f.input)
request := httptest.NewRequest(
    "POST", "/api/bookings", bytes.NewReader(raw))
rec := httptest.NewRecorder()
handler.ServeHTTP(rec, request)
if rec.Code != 201 {
    t.Fatalf("%d %s", rec.Code, rec.Body)
}
```

Первый фрагмент взят из services/schedule/tests/schedule.test.mjs, второй - из services/bookings/service_test.go. Fixture подменяет хранилище, время и внешние HTTP-вызовы. Python-сервисы проверяются pytest и FastAPI TestClient в tests/unit и tests/component.

Покрытие сценариев тестами

Таблица 12 - Матрица unit и компонентных проверок

Сервис	Unit-тесты	Компонентные проверки
S1 Пользователь	Создание, уникальность контакта без учёта регистра, изменение	POST/GET/PATCH, 404/409/422, пустые и неверные поля
S2 Каталог	Уникальный номер, изменение и публикация	Создание/фильтрация/изменение, неподдерживаемый тип игры
S3 Расписание	Непересечение, конкурентные удержания, истечение и повторное закрепление	График, доступность, hold/confirm/release, вызов каталога
S4 Бронирование	Оплата, повтор события, отмена, граница возврата 120/119 минут, поздняя оплата, компенсации	Создание через S1/S5/S3, чтение и отмена, UUID и ошибки
S5 Тарифы	Версии, неизменность цены, точный срок quote, совпадение стола/интервала	Публикация тарифа, quote/validate, вызов каталога
S6 Платежи	Один платёж на бронь, совпадение суммы/валюты, повтор callback и возврат	Токен провайдера, исход оплаты/возврата, ошибки
S7 Сеансы	Единственное посещение, временное окно старта, неявка, автозавершение	Проверка брони через S4, старт/завершение, запрещённые переходы
S8 Уведомления	Дедупликация, отмена напоминания, три попытки, UTC-планирование	Создание/чтение/отправка, получение контакта через S1

Инфраструктурные проверки

Проверены откат состояния/inbox/outbox, повтор события, некорректные UUID и JSON, ошибочный номер попытки, публикация повтора только в очередь своего consumer, отправка в DLQ до ACK и сохранение сообщения при неуспешной публикации.

Конкурентный тест Node.js запускает 32 попытки удержать один интервал; успешно завершается ровно одна. Отдельно проверяются срок quote, граница возврата 120/119 минут и взаимное исключение начала сеанса и неявки.

tests/integration обращается только по HTTP ко всем восьми работающим сервисам Python, Node.js и Go. Проверяются контракты, конкуренция, оплата, отмена и возврат через настоящие PostgreSQL и RabbitMQ. Управляемые границы времени остаются в unit-тестах.

Результаты unit и компонентных тестов

7 ноября 2026 г. выполнены локальные проверки Python, Node.js и Go. Все наборы завершились успешно. В Go подсчитаны функции Test верхнего уровня; вложенные подслучаи повторно не суммируются.

```
python -m pytest tests/unit tests/component --cov
npm --prefix common-node ci
npm --prefix common-node test
go test -race ./...
go vet ./...
```

Таблица 13 - Результаты тестов по языкам

Набор	Результат	Область проверки
Python	91 PASS	86 unit и 5 component; S1/S2/S5/S8, инфраструктура и ПР5
Node.js	75 PASS	S3/S7 и common-node; ошибок и пропусков нет
Go	35 PASS	13 bookings, 9 payments, 13 gokit
Go race / vet	PASS	Проверка гонок данных и статический анализ

Покрытие кода

Python: 78,9% строк в common, practice5 и четырёх сервисах; main.py исключены. API, модели и бизнес-логика сервисов: users 97,5%, catalog 97,7%, pricing 98,1%, notifications 98,5%.

Node.js: 90,38% строк в common-node и API/логике S3/S7; точки входа исключены. Go: bookings 87,3%, payments 86,9%, gokit 42,4% statements; cmd проверяется запуском настоящих контейнеров.

Проценты разных инструментов не объединяются: состав файлов и единицы измерения различаются. У Python одно предупреждение Starlette о будущем изменении TestClient; падений нет.

Журналы в reports: tests.txt, tests.xml, coverage.json, node-tests.txt, go-tests.txt, go-tests.json и go-coverage.out. Команды и ограничения измерения приведены в docs/validation.md.

Проверка реального стенда

На отдельном локальном стенде запущены восемь сервисов, PostgreSQL и RabbitMQ. Выполнен сценарий создания пользователя, стола, графика и тарифа, оформления брони, оплаты, подтверждения, отмены и полного возврата.

Фактический результат scripts.demo


```
{
  "result": "PASS",
  "booking_id": "6ee2b974-d046-42ce-96c0-2c8b0886b5e5",
  "payment_id": "554ae25b-002f-48dc-8bb9-cde319e8f304",
  "booking_status": "cancelled",
  "refund_status": "succeeded",
  "visit_status": "cancelled",
  "interval_available": true,
  "amount_kopecks": 120000
}
```


Таблица 14 - Фактические проверки инфраструктуры

Проверка	Результат
HTTP-контракты и конкуренция	42 PASS; восемь работающих сервисов на трёх языках
Чтение прежних данных	PASS; записи Python доступны через новые S3/S4/S6/S7
Сохранность после down/up	PASS; все 8 контрольных объектов совпали после пересоздания контейнеров
RabbitMQ по вариантам	8 из 8 PASS: четыре профиля × simple/variant
HTTP-операции и контейнеры	33 операции; успешно собраны все 8 приложений

Пересоздание выполнено через полный compose down без -v и compose up -d с сохранёнными томами PostgreSQL и RabbitMQ. Фаза verify прочитала прежние UUID без повторного создания данных: 1 passed. Node.js использует образ версии 24, Go собран компилятором 1.25.

Журналы в reports: integration.txt/xml, demo.json, legacy-data.json, persistence-write.txt, persistence-verify.txt, recreate.txt и rabbitmq.txt. Нагрузочные испытания и кластер БД/брокера не проводились.

Вывод

В практических работах 5-8 реализована система бронирования бильярдных столов из восьми микросервисов. Для каждого участника подготовлен индивидуальный вариант обмена RabbitMQ, а для общей системы определены 13 типов доменных сообщений и 33 HTTP-операции.

Синхронные проверки пользователя, цены, расписания и состояния брони выполняются через HTTP. Оплата, подтверждение, возврат и завершение посещения связываются событиями. Outbox и inbox обеспечивают локальную атомарность и безопасную повторную обработку при доставке как минимум один раз.

Четыре сервиса Python, два Node.js и два Go упакованы в отдельные контейнеры и сохраняют данные в своих базах PostgreSQL. Общий Compose воспроизводит инфраструктуру и приложение. Требование исходящего межсервисного вызова выполнено для всех четырёх участников.

Нативные наборы содержат 201 успешную проверку: 91 Python, 75 Node.js и 35 Go. Дополнительно прошли 42 HTTP-проверки реального стенда, сценарий оплаты и возврата, восемь проверок очередей и чтение всех восьми контрольных объектов после пересоздания контейнеров. Данные, созданные прежней Python-реализацией, также прочитаны новыми сервисами.

Реализация остаётся учебной: использует имитации внешних провайдеров, одну JSON-запись состояния сервиса и ограниченные повторы с DLQ. В отчёте явно указаны границы авторизации, восстановления саги и масштабирования.

Использованные материалы

[1] Методические материалы к практическим работам 5-8 по дисциплине «Микросервисная архитектура». Предоставленный файл manual5-8.pdf. Задания на печатных страницах 32-33, 41, 50 и 55.

[2] Отчёт по практическим работам 1-4 «Бронирование бильярда». Предоставленный файл module1_pr1-4.pdf. Состав команды, границы сервисов и бизнес-правила.

[3] Итоговая текстовая Event Storming диаграмма system-diagram.md. Перечень команд и межсервисных связей.

[4] RabbitMQ Documentation. Consumer Acknowledgements and Publisher Confirms. https://www.rabbitmq.com/docs/confirms. Подтверждения публикации и обработки.

[5] RabbitMQ Documentation. Queues. https://www.rabbitmq.com/docs/queues. Типы, свойства и жизненный цикл очередей.

[6] FastAPI Documentation. Lifespan Events. https://fastapi.tiangolo.com/advanced/events/. Жизненный цикл приложения.

[7] FastAPI Documentation. Testing. https://fastapi.tiangolo.com/tutorial/testing/. Тестирование HTTP-интерфейса.

[8] SQLAlchemy Documentation. Managing Transactions. https://docs.sqlalchemy.org/en/20/orm/session_transaction.html. Локальные транзакции.

[9] Node.js Documentation. Test runner. https://nodejs.org/api/test.html. Встроенные unit- и HTTP-тесты.

[10] Go Documentation. testing и net/http/httptest. https://pkg.go.dev/testing и https://pkg.go.dev/net/http/httptest. Тестирование Go-сервисов.

[11] Исходный код и материалы проекта. https://github.com/weirdflexer/microservices. Контракты, реализация, тесты и электронный отчёт.

Дата обновления отчёта и проверки проекта: 7 ноября 2026 г.